# Python 程式語言

## Lecture 9: Data Analysis Fundamentals

Instructor: 顏家珩 Chia-Heng Yen

# Course Roadmap

- Fundamentals & Environment
  - Introduction to Python and Development Environment Setup
  - Basic Syntax and Flow Control
  - Data Structures and Collections
- Advanced Programming
  - Modular and Functional Programming
  - File Handling and Exception Handling
  - Object-Oriented Programming
  - Algorithm Implementation in Python
- User Interface Development
  - GUI Programming with Tkinter and PyQt
- Data Processing & Analysis
  - Data Analysis Fundamentals
  - Scientific Computing Applications
  - Data Visualization
- AI & ML Applications
  - Introduction to Machine Learning with Applications
  - AI-Related Library in Python Applications

# Course Objectives

- Master NumPy array operations
- Perform data manipulation with Pandas
- Clean and preprocess real-world datasets

# Outline

- NumPy Fundamentals
- Pandas Data Processing
- Data Cleaning

# NumPy Fundamentals

# What is NumPy?

- NumPy = Numerical Python
- Key Features
  - High-performance multidimensional array object
  - Broadcasting capabilities
  - Mathematical and statistical functions
  - Foundation for scientific computing in Python

```
# Python list vs NumPy
import time
import numpy as np
# Python list
python_list = list(range(1000000))
start = time.time()
result = [x * 2 for x in python_list]
print(f"Python list time: {time.time() - start:.4f} seconds")
# NumPy array
numpy_array = np.arange(1000000)
start = time.time()
result = numpy_array * 2
print(f"NumPy array time: {time.time() - start:.4f} seconds")
# NumPy is typically 10-100x faster!
```

# NumPy: Foundation of Data Science

- The Ecosystem Built on NumPy
- NumPy (Data Processing)
  - Pandas (Data Analysis)
  - Matplotlib (Data Visualization)
  - Scikit-learn (Machine Learning)
- Why NumPy is Essential
  - N-dimensional array object
  - Universal functions (ufuncs)
  - Memory efficiency
  - C/Fortran integration

# Installing NumPy

- # Install NumPy
  pip install numpy
  conda install numpy
- # Verify installation
  python -c "import numpy; print(numpy.__version__)"

```
# Import convention
import numpy as np

# Check version
print(np.__version__)
```

# Creating NumPy Arrays

- From Python Lists

```
import numpy as np

# 1D array
arr1d = np.array([1, 2, 3, 4, 5])
print(arr1d)
print(f"Shape: {arr1d.shape}") # (5,)
print(f"Dimension: {arr1d.ndim}") # 1

# 2D array
arr2d = np.array([[1, 2, 3], [4, 5, 6]])
print(arr2d) print(f"Shape: {arr2d.shape}") # (2, 3)
print(f"Dimension: {arr2d.ndim}") # 2

# 3D array
arr3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
print(f"Shape: {arr3d.shape}") # (2, 2, 2)
print(f"Dimension: {arr3d.ndim}") # 3
```

# Creating NumPy Arrays

- Array Creation Functions

```
# Zeros
zeros = np.zeros((3, 4))
print("Zeros array:\n", zeros)
# Ones
ones = np.ones((2, 3))
print("\nOnes array:\n", ones)
# Empty
empty = np.empty((2, 2))
print("\nEmpty array:\n", empty)
# Full
full = np.full((3, 3), 7)
print("\nFull array:\n", full)
# Identity matrix
identity = np.eye(4)
print("\nIdentity matrix:\n", identity)
# Range arrays
arange = np.arange(0, 10, 2) # start, stop, step
print("\nArange:", arange) # [0 2 4 6 8]
linspace = np.linspace(0, 1, 5) # start, stop, num
print("Linspace:", linspace) # [0. 0.25 0.5 0.75 1. ]
```

# Creating NumPy Arrays

- Random Arrays

```
# Set random seed for reproducibility
np.random.seed(42)
# Random integers
rand_int = np.random.randint(0, 10, size=(3, 4))
print("Random integers:\n", rand_int)
# Random floats [0, 1)
rand_float = np.random.random((3, 3))
print("\nRandom floats:\n", rand_float)
# Normal distribution
normal = np.random.randn(3, 3) # mean=0, std=1
print("\nNormal distribution:\n", normal)
# Uniform distribution
uniform = np.random.uniform(0, 10, size=(3, 3))
print("\nUniform distribution:\n", uniform)
# Random choice from array
choices = np.random.choice([1, 2, 3, 4, 5], size=10)
print("\nRandom choices:", choices)
```

# Creating NumPy Arrays

- Array Attributes

```
arr = np.array([   [1, 2, 3, 4],
                   [5, 6, 7, 8],
                   [9, 10, 11, 12]])

print(f"Array:\n{arr}\n")

# Shape
print(f"Shape: {arr.shape}") # (3, 4)
# Size
print(f"Size: {arr.size}") # 12
# Dimensions
print(f"Dimensions: {arr.ndim}") # 2
# Data type
print(f"Data type: {arr.dtype}") # int64
# Item size (bytes)
print(f"Item size: {arr.itemsize} bytes")
# Total size (bytes)
print(f"Total size: {arr.nbytes} bytes")
```

# Array Indexing and Slicing

- Basic Indexing

```
arr = np.array([10, 20, 30, 40, 50])

# Positive indexing
print(arr[0]) # 10
print(arr[2]) # 30

# Negative indexing
print(arr[-1]) #
print(arr[-2]) # 40

# Slicing
print(arr[1:4]) # [20 30 40]
print(arr[:3]) # [10 20 30]
print(arr[2:]) # [30 40 50]
print(arr[::2]) # [10 30 50]
print(arr[::-1]) # [50 40 30 20 10]
```

# Array Indexing and Slicing

- 2D Array Indexing

```
arr2d = np.array([  [1, 2, 3, 4],
                    [5, 6, 7, 8],
                    [9, 10, 11, 12]])
# Single element
print(arr2d[0, 0]) # 1
print(arr2d[1, 2]) # 7
print(arr2d[-1, -1]) # 12
# Row slicing
print(arr2d[0]) # [1 2 3 4]
print(arr2d[1:])
# Column slicing
print(arr2d[:, 0]) # [1 5 9]
print(arr2d[:, 1:3])
# Subarray
print(arr2d[0:2, 1:3])
# Output:
# [[2 3]
# [6 7]]
# Boolean indexing
mask = arr2d > 6
print(arr2d[mask]) # [ 7 8 9 10 11 12]
```

# Array Operations

- Element-wise Operations

```
a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

# Arithmetic operations
print("Addition:", a + b) # [11 22 33 44]
print("Subtraction:", a - b) # [-9 -18 -27 -36]
print("Multiplication:", a * b) # [10 40 90 160]
print("Division:", b / a) # [10. 10. 10. 10.]
print("Power:", a ** 2) # [1 4 9 16]
print("Modulo:", b % a) # [0 0 0 0]

# Comparison operations
print("Greater than:", a > 2) # [False False True True]
print("Equal:", a == 3) # [False False True False]

# Scalar operations
print("Add 10:", a + 10) # [11 12 13 14]
print("Multiply by 2:", a * 2) # [2 4 6 8]
```

# Broadcasting

- Broadcasting allows arrays with different shapes to be operated together

```
# Example 1: Array + Scalar
a = np.array([1, 2, 3])
result = a + 10
print(result) # [11 12 13]
# Example 2: 2D + 1D
arr2d = np.array([[1, 2, 3], [4, 5, 6]])
arr1d = np.array([10, 20, 30])
result = arr2d + arr1d
print(result) # [[11 22 33] # [14 25 36]]
# Example 3: Column vector + Row vector
col = np.array([[1], [2], [3]]) # Shape: (3, 1)
row = np.array([10, 20, 30]) # Shape: (3,)
result = col + row
print(result)     # [[11 21 31]
                  # [12 22 32]
                  # [13 23 33]]
# Visualization of broadcasting
print(f"Column shape: {col.shape}")
print(f"Row shape: {row.shape}")
print(f"Result shape: {result.shape}")
```

# Mathematical Functions

```
arr = np.array([1, 4, 9, 16, 25])
# Basic math
print("Square root:", np.sqrt(arr)) # [1. 2. 3. 4. 5.]
print("Exponential:", np.exp([1, 2, 3])) # [e^1, e^2, e^3]
print("Logarithm:", np.log(arr))
print("Log10:", np.log10(arr))

# Trigonometric
angles = np.array([0, np.pi/2, np.pi])
print("Sine:", np.sin(angles)) # [0. 1. 0.]
print("Cosine:", np.cos(angles)) # [ 1. 0. -1.]
print("Tangent:", np.tan(angles))

# Rounding
values = np.array([1.2, 2.7, 3.5, 4.9])
print("Round:", np.round(values)) # [1. 3. 4. 5.]
print("Floor:", np.floor(values)) # [1. 2. 3. 4.]
print("Ceil:", np.ceil(values)) # [2. 3. 4. 5.]

# Absolute value
negative = np.array([-1, -2, 3, -4])
print("Absolute:", np.abs(negative)) # [1 2 3 4]
```

# Statistical Functions

```
data = np.array([ [1, 2, 3, 4],
                  [5, 6, 7, 8],
                  [9, 10, 11, 12]])

# Basic statistics
print(f"Min: {data.min()}") # 1
print(f"Max: {data.max()}") # 12
print(f"Sum: {data.sum()}") # 78
print(f"Mean: {data.mean()}") # 6.5
print(f"Median: {np.median(data)}") # 6.5
print(f"Std: {data.std()}")
print(f"Variance: {data.var()}")

# Axis-specific operations
print("\nColumn-wise (axis=0):")
print(f"Sum: {data.sum(axis=0)}")
print(f"Mean: {data.mean(axis=0)}")
print("\nRow-wise (axis=1):")
print(f"Sum: {data.sum(axis=1)}")
print(f"Mean: {data.mean(axis=1)}")
```

```
# Percentiles
print(f"\n25th percentile: {np.percentile(data, 25)}")
print(f"75th percentile: {np.percentile(data, 75)}")

# Cumulative operations
print(f"Cumulative sum: {np.cumsum([1, 2, 3, 4])}")
# [1 3 6 10]
```

# Array Reshaping

```
# Original array
arr = np.arange(12)
print("Original:", arr)
# [ 0 1 2 3 4 5 6 7 8 9 10 11]

# Reshape
reshaped = arr.reshape(3, 4)
print("\nReshaped (3x4):\n", reshaped)

# Flatten
flattened = reshaped.flatten()
print("\nFlattened:", flattened)

# Ravel (similar to flatten but returns view)
raveled = reshaped.ravel()
print("Raveled:", raveled)

# Transpose
transposed = reshaped.T
print("\nTransposed:\n", transposed)
```

```
# Add dimension
expanded = arr[:, np.newaxis] # or arr[:, None]
print(f"\nExpanded shape: {expanded.shape}") # (12, 1)

# Squeeze (remove single-dimensional entries)
squeezed = np.squeeze(expanded)
print(f"Squeezed shape: {squeezed.shape}") # (12,)

# Resize (changes original array)
arr.resize(2, 6)
print("\nResized:\n", arr)
```

# Array Concatenation

```
a = np.array([[1, 2],
              [3, 4]])
b = np.array([[5, 6],
              [7, 8]])

# Vertical stacking
vertical = np.vstack([a, b])
print("Vertical stack:\n", vertical)
# [[1 2]
#  [3 4]
#  [5 6]
#  [7 8]]

# Horizontal stacking
horizontal = np.hstack([a, b])
print("\nHorizontal stack:\n", horizontal)
# [[1 2 5 6]
#  [3 4 7 8]]
```

```
# Concatenate with axis
concat_0 = np.concatenate([a, b], axis=0)
# Same as vstack
concat_1 = np.concatenate([a, b], axis=1)
# Same as hstack

print("\nConcatenate axis=0:\n", concat_0)
print("\nConcatenate axis=1:\n", concat_1)

# Column stack
c = np.array([1, 2, 3])
d = np.array([4, 5, 6])
col_stack = np.column_stack([c, d])
print("\nColumn stack:\n", col_stack)
# [[1 4]
#  [2 5]
#  [3 6]]
```

# Array Splitting

```
arr = np.arange(16).reshape(4, 4)
print("Original array:\n", arr)

# Horizontal split
h_split = np.hsplit(arr, 2) # Split into 2 parts
print("\nHorizontal split:")
for i, part in enumerate(h_split):
    print(f"Part {i+1}:\n{part}\n")

# Vertical split
v_split = np.vsplit(arr, 2)
print("Vertical split:")
for i, part in enumerate(v_split):
    print(f"Part {i+1}:\n{part}\n")

# Split at specific indices
arr1d = np.arange(10)
parts = np.split(arr1d, [3, 7]) # Split at index 3 and 7
print("Split at [3, 7]:")
for i, part in enumerate(parts):
    print(f"Part {i+1}: {part}")
```

# Copy vs View

```python
# Original array
original = np.array([1, 2, 3, 4, 5])

# View (shares memory)
view = original.view()
view[0] = 999
print("Original after view change:", original)
# [999 2 3 4 5]
print("View:", view)
# [999 2 3 4 5]

# Reset
original = np.array([1, 2, 3, 4, 5])

# Copy (independent)
copy = original.copy()
copy[0] = 888
print("\nOriginal after copy change:", original)
# [1 2 3 4 5]
print("Copy:", copy)
# [888 2 3 4 5]
```

```python
# Slicing creates view
original = np.array([1, 2, 3, 4, 5])
slice_view = original[1:4]
slice_view[0] = 777
print("\nOriginal after slice change:", original)
# [1 777 3 4 5]

# Check if array owns its data
print(f"\nOriginal owns data: {original.flags['OWNDATA']}")
print(f"View owns data: {view.flags['OWNDATA']}")
print(f"Copy owns data: {copy.flags['OWNDATA']}")
```

# Pandas Data Processing

# What is Pandas?

- Pandas = Panel Data
- Key Features
  - DataFrame: 2D labeled data structure
  - Series: 1D labeled array
  - Built on NumPy for performance
  - Excel-like operations in Python

# Installing pandas

- # Install Pandas
  pip install pandas
  conda install pandas
- # Verify installation
  python -c "import pandas; print(pandas.__version__)"

```
# Import convention
import pandas as pd
import numpy as np

# Check version
print(f"Pandas version: {pd.__version__}")
```

# Series: 1D Labeled Array

```
import pandas as pd
import numpy as np

# Create Series from list
s1 = pd.Series([10, 20, 30, 40, 50])
print("Series from list:\n", s1)

# Create Series with custom index
s2 = pd.Series([10, 20, 30, 40, 50], index=['a', 'b', 'c', 'd', 'e'])
print("\nSeries with custom index:\n", s2)

# Create Series from dictionary
data = {'Math': 95, 'English': 87, 'Science': 92}
s3 = pd.Series(data)
print("\nSeries from dictionary:\n", s3)

# Series attributes
print(f"\nValues: {s3.values}") # NumPy array
print(f"Index: {s3.index}") # Index object
print(f"Data type: {s3.dtype}") # Data type
print(f"Size: {s3.size}") # Number of elements
print(f"Shape: {s3.shape}") # Shape tuple
```

# Series Operations

```python
# Arithmetic operations
s = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
print("Original:", s.values)
print("Add 5:", (s + 5).values)
print("Multiply by 2:", (s * 2).values)
print("Square:", (s ** 2).values)

# Series with Series
s1 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
s2 = pd.Series([4, 5, 6], index=['b', 'c', 'd'])
result = s1 + s2
print("\nSeries addition:\n", result)
# a   NaN
# b   6.0
# c   8.0
# d   NaN
```

```python
# Boolean indexing
print("\nValues > 25:", s[s > 25].values)

# Statistical methods
print(f"\nMean: {s.mean()}")
print(f"Median: {s.median()}")
print(f"Std: {s.std()}")
print(f"Sum: {s.sum()}")
```

# DataFrame: 2D Labeled Data Structure

```
# Method 1: From dictionary
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 28],
    'City': ['New York', 'London', 'Paris', 'Tokyo'],
    'Salary': [50000, 60000, 55000, 58000]
}
df1 = pd.DataFrame(data)
print("DataFrame from dictionary:\n", df1)

# Method 2: From list of dictionaries
data = [
    {'Name': 'Alice', 'Age': 25, 'City': 'New York'},
    {'Name': 'Bob', 'Age': 30, 'City': 'London'},
    {'Name': 'Charlie', 'Age': 35, 'City': 'Paris'}
]
df2 = pd.DataFrame(data)
print("\nDataFrame from list of dicts:\n", df2)
```

```
# Method 3: From NumPy array
data = np.random.randint(0, 100, size=(4, 3))
df3 = pd.DataFrame(
            data,
            columns=['Math', 'English', 'Science'],
            index=['Student1', 'Student2', 'Student3',
'Student4'])
print("\nDataFrame from NumPy array:\n", df3)
```

# DataFrame Attributes

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [25, 30, 35, 28, 32],
    'City': ['NY', 'LA', 'Chicago', 'Boston', 'Seattle'],
    'Salary': [50000, 60000, 55000, 58000, 62000]
})
# Basic information
print("Shape:", df.shape) # (rows, columns)
print("Size:", df.size) # Total elements
print("Columns:", df.columns.tolist()) # Column names
print("Index:", df.index.tolist()) # Row indices
print("Data types:\n", df.dtypes) # Column data types

# First and last rows
print("\nFirst 3 rows:\n", df.head(3))
print("\nLast 2 rows:\n", df.tail(2))

# Statistical summary
print("\nStatistical summary:\n", df.describe())

# Information about DataFrame
print("\nDataFrame info:")
df.info()
```

# Data Selection: Columns

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['NY', 'LA', 'Chicago'],
    'Salary': [50000, 60000, 55000]
})

# Single column (returns Series)
names = df['Name']
print("Names (Series):\n", names)
print(f"Type: {type(names)}")

# Single column as DataFrame
names_df = df[['Name']]
print("\nNames (DataFrame):\n", names_df)
print(f"Type: {type(names_df)}")
```

```
# Multiple columns
subset = df[['Name', 'Salary']]
print("\nName and Salary:\n", subset)

# Add new column
df['Bonus'] = df['Salary'] * 0.1
print("\nWith Bonus column:\n", df)

# Derived column
df['Total_Compensation'] = df['Salary'] + df['Bonus']
print("\nWith Total Compensation:\n", df)

# Delete column
df_dropped = df.drop('Bonus', axis=1)
# axis=1 for columns
print("\nAfter dropping Bonus:\n", df_dropped)
```

# Data Selection: Rows

- Using loc (label-based)

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 28],
    'Salary': [50000, 60000, 55000, 58000]
}, index=['emp1', 'emp2', 'emp3', 'emp4'])

# Single row by label
print("Employee 2:\n", df.loc['emp2'])

# Multiple rows
print("\nEmployees 2-3:\n", df.loc['emp2':'emp3'])

# Rows and columns
print("\nNames and Ages of emp1-emp2:\n",
    df.loc['emp1':'emp2', ['Name', 'Age']])

# Boolean indexing with loc
high_salary = df.loc[df['Salary'] > 55000]
print("\nHigh salary employees:\n", high_salary)
```

# Data Selection: iloc (position-based)

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 28],
    'Salary': [50000, 60000, 55000, 58000]
})

# Single row by position
print("First row:\n", df.iloc[0])
# Multiple rows
print("\nFirst 2 rows:\n", df.iloc[0:2])
# Specific rows and columns
 print("\nRows 1-2, Columns 0-1:\n", df.iloc[1:3, 0:2])
# Last row
print("\nLast row:\n", df.iloc[-1])
# Every other row
print("\nEvery other row:\n", df.iloc[::2])
# Specific positions
print("\nRows 0 and 2, Columns 1 and 2:\n", df.iloc[[0, 2], [1, 2]])
```

# Boolean Indexing

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [25, 30, 35, 28, 32],
    'City': ['NY', 'LA', 'Chicago', 'Boston', 'Seattle'],
    'Salary': [50000, 60000, 55000, 58000, 62000]
})

# Single condition
young = df[df['Age'] < 30]
print("Employees younger than 30:\n", young)

# Multiple conditions with & (AND)
young_high_salary = df[(df['Age'] < 35) &
(df['Salary'] > 55000)]
print("\nYoung AND high salary:\n",
young_high_salary)

# Multiple conditions with | (OR)
young_or_high = df[(df['Age'] < 28) |
(df['Salary'] > 60000)]
print("\nYoung OR high salary:\n", young_or_high)
```

```
# NOT condition
not_ny = df[~(df['City'] == 'NY')]
print("\nNot in NY:\n", not_ny)

# isin() method
cities = df[df['City'].isin(['NY', 'LA'])]
print("\nFrom NY or LA:\n", cities)

# String methods
starts_with_c = df[df['Name'].str.startswith('C')]
print("\nNames starting with 'C':\n", starts_with_c)
```

# Reading and Writing Data

- CSV Files

```
# Create sample data
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['NY', 'LA', 'Chicago'],
    'Salary': [50000, 60000, 55000]
})

# Write to CSV
df.to_csv('employees.csv', index=False)
print("Saved to employees.csv")

# Read from CSV
df_read = pd.read_csv('employees.csv')
print("\nRead from CSV:\n", df_read)
```

```
# Read with custom options
df_custom = pd.read_csv(  'employees.csv',
                          encoding='utf-8',
                          sep=',',
                          header=0, n
                          ames=['Name', 'Age', 'City', 'Salary'])

# Read specific columns
df_subset = pd.read_csv('employees.csv',
                                usecols=['Name', 'Salary'])
print("\nSubset columns:\n", df_subset)

# Read with conditions
df_filtered = pd.read_csv('employees.csv',
                                  skiprows=lambda x: x > 0 and x % 2 == 0)
print("\nFiltered rows:\n", df_filtered)
```

# Excel Files

```
# Write to Excel
# Requires: pip install openpyxl
df.to_excel('employees.xlsx',
            sheet_name='Staff',
            index=False)
print("Saved to employees.xlsx")

# Read from Excel
 df_excel = pd.read_excel('employees.xlsx', sheet_name='Staff')
print("\nRead from Excel:\n", df_excel)

# Write multiple sheets
with pd.ExcelWriter('company.xlsx') as writer:
    df.to_excel(writer, sheet_name='Employees', index=False)
    df[df['Age'] > 30].to_excel(writer, sheet_name='Senior', index=False)

print("Created multi-sheet Excel file")

# Read all sheets
all_sheets = pd.read_excel('company.xlsx', sheet_name=None)
print(f"\nSheet names: {list(all_sheets.keys())}")
```

# JSON Files

```
# Write to JSON
df.to_json('employees.json', orient='records', indent=2)
print("Saved to employees.json")

# Read from JSON
df_json = pd.read_json('employees.json')
print("\nRead from JSON:\n", df_json)

# Different orientations
df.to_json('employees_split.json', orient='split', indent=2)
df.to_json('employees_index.json', orient='index', indent=2)
df.to_json('employees_columns.json', orient='columns', indent=2)
df.to_json('employees_values.json', orient='values', indent=2)
print("\nCreated JSON files with different orientations")
```

# Data Sorting

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Age': [25, 30, 35, 28, 32],
    'Salary': [50000, 60000, 55000, 58000, 62000],
    'Department': ['IT', 'HR', 'IT', 'Finance', 'HR']
})

# Sort by single column
sorted_age = df.sort_values('Age')
print("Sorted by Age:\n", sorted_age)

# Sort descending
sorted_salary_desc = df.sort_values('Salary',
ascending=False)
print("\nSorted by Salary (descending):\n",
sorted_salary_desc)

# Sort by multiple columns
sorted_multi = df.sort_values(['Department',
'Salary'], ascending=[True, False])
print("\nSorted by Department then Salary:\n",
sorted_multi)
```

```
# Sort by index
df_custom_index = df.set_index('Name')
sorted_index = df_custom_index.sort_index()
print("\nSorted by Name (index):\n", sorted_index)

# Inplace sorting
df_copy = df.copy()
df_copy.sort_values('Age', inplace=True)
print("\nSorted inplace:\n", df_copy)
```

# Data Aggregation

```
df = pd.DataFrame({
    'Department': ['IT', 'IT', 'HR', 'HR', 'Finance'],
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'Salary': [50000, 60000, 55000, 58000, 62000],
    'Age': [25, 30, 35, 28, 32]
})
# Group by single column
grouped = df.groupby('Department')

# Aggregate functions
print("Mean salary by department:\n",
grouped['Salary'].mean())
print("\nSum salary by department:\n",
grouped['Salary'].sum())
print("\nCount by department:\n", grouped.size())

# Multiple aggregations
agg_result = grouped['Salary'].agg(['mean', 'min',
'max', 'std'])
print("\nMultiple aggregations:\n", agg_result)
```

```
# Different functions for different columns
multi_agg = grouped.agg({ 'Salary': ['mean', 'sum'],
'Age': ['min', 'max'] })
print("\nMultiple columns aggregation:\n",
multi_agg)

# Apply custom function
def salary_range(x):
    return x.max() - x.min()
range_result = grouped['Salary'].agg(salary_range)
print("\nSalary range by department:\n",
range_result)
```

# Data Merging and Joining

```
# Sample DataFrames
employees = pd.DataFrame({
    'EmployeeID': [1, 2, 3, 4],
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'DeptID': [10, 20, 10, 30]
})
departments = pd.DataFrame({
    'DeptID': [10, 20, 30, 40],
    'DeptName': ['IT', 'HR', 'Finance', 'Marketing']
})
# Inner join (intersection)
inner = pd.merge(employees, departments,
on='DeptID', how='inner')
print("Inner Join:\n", inner)

# Left join
left = pd.merge(employees, departments,
on='DeptID', how='left')
print("\nLeft Join:\n", left)

# Right join
right = pd.merge(employees, departments,
on='DeptID', how='right')
print("\nRight Join:\n", right)
```

```
# Outer join (union)
outer = pd.merge(employees, departments,
on='DeptID', how='outer')
print("\nOuter Join:\n", outer)

# Concatenate vertically
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
concat_v = pd.concat([df1, df2], ignore_index=True)
print("\nVertical concatenation:\n", concat_v)

# Concatenate horizontally
concat_h = pd.concat([df1, df2], axis=1)
print("\nHorizontal concatenation:\n", concat_h)
```

# Data Cleaning

# Why Data Cleaning?

- Real-world data is messy!
- Common Issues
  - Missing values
  - Duplicate records
  - Incorrect data types
  - Outliers
  - Inconsistent formatting
  - Invalid entries

```
# Example of messy data
messy_data = pd.DataFrame({
    'Name': ['Alice', 'Bob', None, 'David', 'Eve', 'Bob'],
    'Age': [25, 30, 35, '28', 32, 30],
    'Salary': [50000, 60000, None, 58000, '62000', 60000],
    'Email': ['alice@email.com', 'bob@email', 'charlie@email.com', 'david@email.com', 'eve@email.com',
'bob@email']
})

print("Messy Data:\n", messy_data)
print("\nData Info:")
messy_data.info()
```

# Handling Missing Values

- Detecting Missing Values

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', None, 'David', 'Eve'],
    'Age': [25, 30, 35, None, 32],
    'Salary': [50000, None, 55000, 58000, 62000],
    'City': ['NY', 'LA', 'Chicago', None, 'Seattle']
})

# Check for missing values
print("Missing values:\n", df.isnull())
print("\nMissing values sum:\n", df.isnull().sum())
print("\nTotal missing:", df.isnull().sum().sum())
# Check for non-missing values
print("\nNon-missing values:\n", df.notnull().sum())
# Percentage of missing values
missing_percent = (df.isnull().sum() / len(df)) * 100
print("\nMissing percentage:\n", missing_percent)
```

# Handling Missing Values

- Removing Missing Values

```
df = pd.DataFrame({
    'A': [1, 2, None, 4],
    'B': [5, None, None, 8],
    'C': [9, 10, 11, 12]
})
print("Original:\n", df)

# Drop rows with any missing value
df_dropna_any = df.dropna()
print("\nDrop rows with ANY missing:\n",
df_dropna_any)

# Drop rows where all values are missing
df_all_missing = pd.DataFrame({
    'A': [1, None, 3],
    'B': [None, None, 6],
    'C': [7, None, 9]
})
df_dropna_all = df_all_missing.dropna(how='all')
print("\nDrop rows with ALL missing:\n",
df_dropna_all)

# Drop rows with missing in specific columns
df_dropna_subset = df.dropna(subset=['A'])
print("\nDrop rows missing in column 'A':\n",
df_dropna_subset)

# Drop columns with missing values
df_dropna_cols = df.dropna(axis=1)
print("\nDrop columns with missing:\n",
df_dropna_cols)

# Threshold: keep rows with at least N non-null values
df_thresh = df.dropna(thresh=2) # At least 2 non-null
print("\nKeep rows with at least 2 non-null:\n",
df_thresh)
```

# Handling Missing Values

- Filling Missing Values

```
df = pd.DataFrame({
    'A': [1, 2, None, 4, None],
    'B': [5, None, 7, None, 9],
    'C': [10, 11, 12, 13, 14]
})
print("Original:\n", df)
# Fill with a constant
df_fill_0 = df.fillna(0)
print("\nFill with 0:\n", df_fill_0)

# Fill with mean
df_fill_mean = df.fillna(df.mean())
print("\nFill with mean:\n", df_fill_mean)

# Fill with median
df_fill_median = df.fillna(df.median())
print("\nFill with median:\n", df_fill_median)

# Forward fill (use previous value)
df_ffill = df.fillna(method='ffill')
print("\nForward fill:\n", df_ffill)
# Backward fill (use next value)
df_bfill = df.fillna(method='bfill')
print("\nBackward fill:\n", df_bfill)

# Fill different columns with different values
df_fill_dict = df.fillna({'A': 0, 'B': df['B'].mean()})
print("\nFill with dict:\n", df_fill_dict)

# Interpolate
df_interpolate = df.interpolate()
print("\nInterpolate:\n", df_interpolate)
```

# Handling Duplicates

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'Bob', 'Alice'],
    'Age': [25, 30, 35, 30, 25],
    'City': ['NY', 'LA', 'Chicago', 'LA', 'NY']
})
print("Original:\n", df)

# Check for duplicates
print("\nDuplicate rows:\n", df.duplicated())
print("\nNumber of duplicates:", df.duplicated().sum())

# Find duplicate rows
duplicates = df[df.duplicated()]
print("\nDuplicate records:\n", duplicates)

# Check duplicates based on specific columns
dup_name = df.duplicated(subset=['Name'])
print("\nDuplicate names:\n", dup_name)

# Remove duplicates
df_no_dup = df.drop_duplicates()
print("\nAfter removing duplicates:\n", df_no_dup)

# Keep last occurrence
df_keep_last = df.drop_duplicates(keep='last')
print("\nKeep last duplicate:\n", df_keep_last)

# Remove duplicates based on specific columns
df_no_dup_name = df.drop_duplicates(subset=['Name'])
print("\nRemove duplicate names:\n", df_no_dup_name)

# Remove all duplicates
df_no_dup_all = df.drop_duplicates(keep=False)
print("\nRemove all duplicates:\n", df_no_dup_all)
```

# Data Type Conversion

```
df = pd.DataFrame({
    'ID': ['1', '2', '3', '4'],
    'Age': ['25', '30', '35', '28'],
    'Salary': ['50000', '60000', '55000', '58000'],
    'Is_Active': ['True', 'False', 'True', 'True']
})
print("Original data types:")
print(df.dtypes)
print("\nOriginal:\n", df)

# Convert to numeric
df['ID'] = pd.to_numeric(df['ID'])
df['Age'] = pd.to_numeric(df['Age'])
df['Salary'] = pd.to_numeric(df['Salary'])

print("\nAfter numeric conversion:")
print(df.dtypes)

# Convert to boolean
df['Is_Active'] = df['Is_Active'] == 'True'
print("\nWith boolean conversion:")
print(df.dtypes)
print(df)
```

```
# Handle conversion errors
df_with_errors = pd.DataFrame({
    'Values': ['1', '2', 'three', '4', 'five']
})

# coerce: invalid values become NaN
df_with_errors['Values_numeric'] = pd.to_numeric(
    df_with_errors['Values'], errors='coerce'
)
print("\nWith coerce errors:\n", df_with_errors)

# Convert to datetime
df_dates = pd.DataFrame({
    'Date': ['2024-01-01', '2024-02-15', '2024-03-30']
})
df_dates['Date'] = pd.to_datetime(df_dates['Date'])
print("\nDateTime conversion:")
print(df_dates.dtypes)
```

# String Cleaning

```
df = pd.DataFrame({
    'Name': [' Alice ', 'BOB', 'charlie', ' David'],
    'Email': ['alice@GMAIL.com', 'bob@email.COM', 'charlie@email.com', 'david@EMAIL.com']
})
print("Original:\n", df)

# Remove whitespace
df['Name_clean'] = df['Name'].str.strip()
print("\nAfter strip:\n", df[['Name', 'Name_clean']])

# Convert to lowercase
df['Name_lower'] = df['Name'].str.lower().str.strip()
print("\nLowercase:\n", df[['Name', 'Name_lower']])

# Convert to uppercase
df['Name_upper'] = df['Name'].str.upper().str.strip()

# Title case
df['Name_title'] = df['Name'].str.strip().str.title()
print("\nTitle case:\n", df[['Name', 'Name_title']])
```

```
# Clean email addresses
df['Email_clean'] = df['Email'].str.lower().str.strip()
print("\nCleaned emails:\n", df[['Email', 'Email_clean']])

# Replace text
df['Email_domain'] = df['Email_clean'].str.replace('gmail', 'email')
print("\nReplaced domain:\n", df[['Email_clean', 'Email_domain']])

# Extract parts
df['Username'] = df['Email_clean'].str.split('@').str[0]
df['Domain'] = df['Email_clean'].str.split('@').str[1]
print("\nExtracted parts:\n", df[['Email_clean', 'Username', 'Domain']])
```

# Outlier Detection

- Using Statistical Methods

```
import numpy as np

# Create data with outliers
np.random.seed(42)
normal_data = np.random.normal(50, 10, 95)
outliers = np.array([150, 200, -50, -100, 180])
data = np.concatenate([normal_data, outliers])

df = pd.DataFrame({'Value': data})

# Method 1: Z-Score
df['Z_Score'] = (df['Value'] - df['Value'].mean()) / df['Value'].std()
outliers_z = df[np.abs(df['Z_Score']) > 3]
print("Outliers using Z-Score (|z| > 3):")
print(outliers_z)

# Method 2: IQR (Interquartile Range)
Q1 = df['Value'].quantile(0.25)
Q3 = df['Value'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
outliers_iqr = df[(df['Value'] < lower_bound) |
(df['Value'] > upper_bound)]
print(f"\nOutliers using IQR method:")
print(f"Lower bound: {lower_bound:.2f}")
print(f"Upper bound: {upper_bound:.2f}")
print(outliers_iqr)

# Remove outliers
df_clean = df[(df['Value'] >= lower_bound) &
(df['Value'] <= upper_bound)]
print(f"\nOriginal size: {len(df)}")
print(f"After removing outliers: {len(df_clean)}")
print(f"Removed: {len(df) - len(df_clean)} outliers")

# Visualize statistics
print(f"\nStatistics before cleaning:")
print(df['Value'].describe())
print(f"\nStatistics after cleaning:")
print(df_clean['Value'].describe())
```

# Data Normalization

- Scaling Techniques

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler
df = pd.DataFrame({
    'Feature1': [1, 5, 10, 15, 20],
    'Feature2': [100, 200, 300, 400, 500],
    'Feature3': [0.1, 0.5, 1.0, 1.5, 2.0]
})
print("Original data:\n", df)
print("\nOriginal statistics:\n", df.describe())

# Min-Max Normalization (0 to 1)
scaler_minmax = MinMaxScaler()
df_minmax = pd.DataFrame(
    scaler_minmax.fit_transform(df),
    columns=df.columns
)
print("\nMin-Max Normalized (0-1):\n", df_minmax)
```

```
# Z-Score Normalization (mean=0, std=1)
scaler_standard = StandardScaler()
df_standard = pd.DataFrame(
    scaler_standard.fit_transform(df),
    columns=df.columns
)
print("\nStandardized (Z-Score):\n", df_standard)
print("\nStandardized statistics:\n",
df_standard.describe())

# Manual Min-Max
df_manual_minmax = (df - df.min()) / (df.max() –
df.min())
print("\nManual Min-Max:\n", df_manual_minmax)

# Manual Z-Score
df_manual_zscore = (df - df.mean()) / df.std()
print("\nManual Z-Score:\n", df_manual_zscore)
```